

Exercise 1

The following program illustrates the behavior of the fork.

```
void main () {
int code_retour ;
printf ("Debut du test fork ()\n");
code_retour = fork ();
switch (code_retour ) {
case -1 :
printf ("Pbm lors de la creation du processus\n");
break;
case 0 :
printf ("Je suis le processus fils \n");
break;
default :
printf ("Je suis le processus père\n");
printf ("Je viens de créer le processus fils dont le pid est
%d \n", code_retour);
break;
}
printf ("code_retour %d\n", code_retour);
printf ("Fin du test fork ()\n");
}
```

Exécuter ce programme puis expliquer le résultat obtenu.

Exercise 2

Using the fork () procedure and using the getuid, getgid, ... primitives provided in progress, write a program which creates a new process and which displays for the two processes (father and child) the general characteristics of a process :

- Process identifier.
- Identifier of the process parent.
- Working directory of the process.
- The actual owner.
- The beneficial owner.
- The actual owner group.
- The effective owner group.

Before displaying this information, we will take care to specify whether we are in the parent or child process.

Exercise 3

Let's go back to the previous exercise and set a wait of a few seconds (sleep (5)) in the child process so that the parent process ends before the child.

What can we say about the result obtained?

Exercise 4

The purpose of this exercise is to illustrate how a child process inherits a copy of file descriptors opened by its father.

Let's take the frame of the previous exercise and add to it, before creating a new process, the opening of a read / write "foo" file.

Either the "foo" file containing the alphabet.

```
$ cat toto
```

```
Abcdefghijklmnopqrstuvwxyz
```

Write the program that performs the following sequence:

1. The child process begins by writing "Child" to the file and then falls asleep (sleep (2)).
2. The parent process first falls asleep (sleep (1)) then reads 4 characters from the file and displays them on the screen.
3. The parent process then writes "Father" to the file and terminates.
4. After falling asleep (see step 1), the child process reads 4 characters from the file and displays them on the screen and then ends.

Explain the result obtained on the screen and in the file, detailing step by step the sequencing as well as the effect of the various instructions.

Note :

Be careful not to omit the <fcntl.h> file in the inclusion files.

Exercise 5

Starting again from the program of exercise 3, add the instructions that will allow the parent process to wait for its child to terminate and which will display the return code of the latter. We first assume that the child ends correctly: exit (0).

Exercise 6

a) Starting again from the program of the previous exercise, add the instructions that will allow the parent process to interpret the termination information of its child process:

1. Return code of the child process
2. Signal that caused the termination of the child process.

We will terminate the child process with a return code of 3.

b) Then put the child process to sleep (sleep (10)) and then kill it using the shell command: kill -15 pid_children.

We will take care to launch the program in the background.

Interpret the result obtained.

c) Reproduce the previous case with "kill -9 pid_fils".

Exercise N°7

Starting from the program of the previous exercise, add the instructions that will allow the parent process to interpret the termination information of its child process:

- The parent process must create 2 child processes.
- The first child process will fall asleep for 5 seconds
- will display a message before falling asleep and then before ending
- will return the value 1 in return code
- The second child process
- will fall asleep for 10 seconds
- will display a message before falling asleep and then before ending
- will return the value 2 in return code
- The parent process will wait for the second child process to terminate and
- will display its termination information.

Exercise 8

Write a program that runs the "sleep 60" shell command using the system primitive.

Start the corresponding executable in the background and type the command 'ps -l'.

Explain the result obtained.

Perform the same operation by running the shell command in the background: sleep 60 &. What's going on ?

Exercise 9

The following program illustrates the behavior of exec.

```
#include <stdio.h>
#include <unistd.h>
main () {
    char buf [80];
    printf ("%d)-- Debut du test exec () \n",getpid());
    sprintf (buf,"-- pid=%d", getpid());
    execl ("/bin/echo","echo","Execution","d'un test exec",buf, NULL);
    printf ("%d)-- Echec de la fonction execl () \n",getpid());
    printf ("%d)-- Fin du test exec () \n",getpid());
    exit (0);
}
```

a) Perform then explain the result obtained.

b) Replace the command "/ bin / echo" by "/ bin / ech". Explain the result obtained.

Exercise 10

a) Using the fork and execl primitives, write the program that lets you execute the ps command with the -l option from an existing process.

The latter will have to wait for the return code of the ps command.

b) Resume the previous program but using the execv command.

Exercise 11

Après compilation de exo1.c, on exécutera le programme exo1 plusieurs fois. Pourquoi cinq lignes sont-elles affichées alors qu'il n'y a que deux printf dans le programme?

Exo1.c

```
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h> int main (){
    pid_t valeur;
    printf("printf-0 Avant fork : ici processus numero %d\n",(int) getpid());
    valeur = fork();
    printf("printf-1 Valeur retournee par la fonction fork: %d\n", (int)valeur);
    printf("printf-2 Je suis le processus numero %d\n", (int)getpid());
    return 0;
}
```

Now add the following line behind the fork call: if (value == 0) sleep (4); What's going on ? Why ?

Exercise 12

Either the following program: fork-mul.c

```
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>
int main () {
    pid_t valeur, valeur1 ;
    printf(" print 1 - Je suis le processus pere num=%d \n",(int)getpid() );
    valeur = fork();
    printf (" print 2 - retour fork: %d - processus num= %d -num pere=%d \n",(int)valeur,
    (int)getpid(), (int)getppid() );
    valeur1 = fork();
```

```
printf (" print 3 - retour fork: %d - processus num= %d -num pere=%d \n", (int)valeur1,
(int)getpid(), (int)getppid() );
return 0;
}
```

After execution and using the following diagram, note the numbers of the processes and number the execution order of the printf instructions in order to find the execution order of the processes.

Note:

if we note a process number equal to 1, it is the process init, father of all the processes. Init adopts orphaned processes, i.e. a process whose father terminated becomes child of 1, and getppid () returns 1.

Exercise 13

Write a program such as the family tree of the processes generated by its execution and a:

- binary tree of depth 1
- binary tree of depth 2
- binary tree of depth 3

Exercise 14

Consider the following program:

```
#include<unistd.h>
main( ) {
fork();
fork();
fork();
}
```

1. Without performing the execution, give the number of processes generated and draw the tree of the processes generated.
2. Replace the first call to "fork ()" with a call to "execl ()" which executes the program itself.

What's going on ?

3. Replace the second call to "fork ()" with a call to "execl ()" which executes the program itself.

What is happening now?

Exercise 15

Consider the 3 programs defined below.

1. Without executing them, draw the family tree generated by their executions.
2. Implement and run these programs to compare results.

```
#include<unistd.h>
main() {
fork() && fork() ;
}
#include<unistd.h>
main() {
( fork() || fork() ) ;
}
#include<unistd.h>
main() {
fork()&&( fork() || fork() ) ;
sleep(2)
}
```